Massively Parallel Multi-City Transit Schedule Analytics with MPI, MPI-I/O, and CUDA

Shimu Pan
Rensselaer Polytechnic Institute
Troy, New York, USA
pans@rpi.edu

Tim Liakh
Rensselaer Polytechnic Institute
Troy, New York, USA
liakht@rpi.edu

Nikul Patel
Rensselaer Polytechnic Institute
Troy, New York, USA
pateln14@rpi.edu

Abstract

Public transit agencies publish large General Transit Feed Specification (GTFS) datasets that encode routes, trips, calendars, and stop-level schedules for complex multi-modal systems. Although the format is standardized, large-scale analysis is still cumbersome because even basic questions require joining several files, normalizing large numbers of records, and aggregating patterns across cities. This project builds a parallel pipeline for multi-city transit schedule analysis using MPI, MPI-I/O, and CUDA on the AiMOS supercomputer. The pipeline downloads current and archive-backed GTFS feeds from several northeastern and midwestern providers, converts them into a compact fixed-width binary format, and reads them collectively with `MPI_File_read_at_all`. MPI ranks exchange boundary records so headway calculations remain correct across partitions, then reduce provider-level statistics globally. Within each rank, either a CPU-only MPI backend or a CUDA backend computes hourly departures, trip-duration summaries, and headway-related metrics. The resulting dataset contains 3,703,385 trip records across MTA, CTA, MBTA, NJ Transit, and SEPTA. On AiMOS, the CPU path shows useful strong scaling from 50.6 ms at two ranks to 20.4 ms at sixteen ranks on the full dataset and reasonably stable weak-scaling behavior as total work increases from 300,000 to 2.4 million records. The GPU kernel scales locally, but end-to-end runtime remains dominated by fixed CUDA overhead, so the hybrid MPI+CUDA path is slower than CPU-only MPI for this workload.

Keywords

HPC, MPI, MPI-I/O, CUDA, GTFS, transit analytics, strong scaling, weak scaling

1 Introduction

Public transit systems publish schedule data at a scale that is awkward for interactive analysis and expensive to normalize repeatedly. A single GTFS feed is a relational bundle of routes, trips, stop times, stops, calendars, and metadata. In practice, a multi-city study means integrating many agency feeds, handling format quirks, deriving route-level and trip-level features, and then aggregating them into system-level statistics. This fits HPC well: the input is large, preprocessing is expensive, and the final analysis needs both local computation and global coordination.

This project targets that problem with a parallel system built around three steps. First, we convert multi-file GTFS schedules into a fixed-width binary trip-record dataset that supports collective file access. Second, we use MPI and MPI-I/O to distribute that dataset across ranks, exchange boundary state, and reduce global aggregates. Third, we accelerate local analytics with CUDA so that each rank can use the GPU for bucketized schedule statistics and headway accumulation. The resulting workflow is not embarrassingly parallel because route headways cross partition boundaries, city-level statistics require collective reduction, and the input itself is read from a shared parallel file using MPI-I/O.

The application focus is *scheduled service analysis*, not ridership analysis. Because the current pipeline uses static GTFS schedules rather than fare-card, passenger-count, or GTFS-Realtime delay data, the findings in this report concern service intensity, service mix, trip duration, and scheduled headway regularity. Those are still useful operational indicators, and they produce a workload with real communication and reduction costs.

The main contributions of this project are:

- a GTFS preprocessing pipeline that converts heterogeneous public feeds into one compact binary dataset suitable for collective MPI-I/O;
- a distributed analytics engine that combines MPI partitioning, boundary exchange, reduction, and CUDA local processing;
- a provider-level transit analysis across five major agencies using real GTFS schedule data; and
- an experimental design for strong scaling, weak scaling, and CPU-versus-GPU comparison on AiMOS.

2 Related Work

GTFS has become a standard substrate for transit network analysis. The official GTFS schedule and GTFS-Realtime specifications define the semantics and field structure for static and live transit data [6–8]. Those specifications make ecosystem-wide tooling possible, but they do not remove the underlying cost of integrating schedule, calendar, and stop-time files into analytical features.

Several prior works show the analytical value of GTFS. Fayyaz et al. [2] developed an efficient GTFS-enabled algorithm for dynamic transit accessibility analysis. Fortin et al. [3] used GTFS data to build graph-oriented transit network indicators, while Guthrie et al. [4] used GTFS for accessibility scenario analysis in sketch planning. Pereira et al. [12] introduced a multimodal routing framework that relies on GTFS-style network data and supports accessibility-oriented queries at scale.

More recent work uses GTFS in interactive and operational settings. Devunuri and Lehe [1] showed that GTFS datasets can support flexible analytical querying through large language model interfaces. Javanmard et al. [5] and Zhou et al. [14] use GTFS-Realtime for uncertainty-aware accessibility and operational decision support. Together, these studies point to the need for scalable infrastructure that can process multiple feeds repeatedly.

On the HPC side, we rely on standard distributed and accelerator programming models. The MPI 4.1 standard provides the collective communication and MPI-I/O interfaces used by our code [9].

Table 1: Archive-backed GTFS dataset summary after preprocessing. Route counts are snapshot-aware because the dataset includes multiple schedule vintages.

Provider	Trip Records	Unique Routes
MTA	1,092,298	1,708
CTA	319,822	524
MBTA	1,693,567	4,419
NJ Transit	36,483	279
SEPTA	561,215	2,178
Total	3,703,385	9,108

Thakur et al. [13] describe the collective-I/O mechanisms in ROMIO that motivate our use of `MPI_File_read_at_all` over a shared binary dataset. For GPU acceleration, the implementation follows the CUDA programming model described by Nickolls et al. [10] and NVIDIA's programming guide [11]. Our contribution is to apply these standard HPC tools to a transit-schedule workload with shared input, boundary-dependent headways, and provider-level aggregation.

3 Dataset and Problem Formulation

3.1 Provider Set

The current dataset covers five provider groups assembled from publicly accessible GTFS schedule feeds:

- MTA (New York): subway, commuter rail, and several bus subfeeds,
- CTA (Chicago),
- MBTA (Boston),
- NJ Transit, and
- SEPTA (Philadelphia).

The preprocessing stage now produces a consolidated binary dataset of 3,703,385 trip records, each exactly 48 bytes. The larger input comes from combining current feeds with archive-backed schedule snapshots where they were available. Table 1 summarizes the resulting provider-level record counts together with the snapshot-aware route counts derived during preprocessing.

3.2 Trip-Record Representation

The preprocessing stage converts each GTFS trip into a fixed-width record:

```
(city_id, feed_id, route_type, route_hash, service_mask,
direction_id, start_secs, end_secs, duration_secs,
stop_count,
trip_hash, reserved)
```

This representation is intentionally compact. It avoids expensive string parsing during parallel execution and allows the MPI layer to partition input by record count rather than by text-file boundaries.

Records are sorted by:

```
(city_id, route_hash, direction_id, start_secs,
trip_hash)
```

That sort order is critical because the route-headway computation depends on adjacent records within each city, route, and direction.

Pipeline: GTFS download → preprocessing into fixed-width binary records → collective MPI-I/O read → rank-boundary exchange → CPU or CUDA local analytics → MPI reduction → provider-level summaries and performance reports.

Figure 1: High-level workflow implemented in this project.

3.3 Analytical Questions

This report asks two sets of questions:

(1) **HPC questions**: How does the hybrid MPI+CUDA implementation behave under strong scaling, weak scaling, and CPU-versus-GPU comparison? How much time is spent on file I/O, communication, reduction, and compute?
(2) **Transit questions**: How do providers differ in peak/off-peak concentration, scheduled headway structure, hourly service profile, and mode composition?

These two question sets are linked. Faster analysis makes it easier to compare providers across larger datasets and finer schedule detail.

4 Parallel Algorithm - CUDA and MPI

4.1 Pipeline Overview

Figure 1 shows the end-to-end workflow.

4.2 Preprocessing

The preprocessing stage is implemented in Python. It reads `routes.txt`, `trips.txt`, `stop_times.txt`, and calendar metadata and then derives one record per trip. During implementation, several real-world GTFS packaging differences had to be handled explicitly:

- some feeds include `calendar_dates.txt` but no `calendar.txt`;
- some providers package multiple nested GTFS archives inside one top-level zip; and
- route-type values are not always limited to the canonical small GTFS set, so extended values must later be normalized for reporting.

A preprocessing pipeline for real feeds has to handle those packaging differences.

4.3 MPI-I/O and Partitioning

At runtime, the analyzer opens the binary dataset collectively with `MPI_File_open`, obtains the file size, converts it to a record count, and partitions the record range across ranks. Each rank then issues a collective `MPI_File_read_at_all` to fetch its contiguous record block.

This avoids a serial rank-0 read followed by a broadcast or scatter. It also puts the input path inside the measured workload, which matters here because I/O is a real part of the runtime.

4.4 Boundary Exchange

Route headways cross partition boundaries. If one rank ends with trip t_i and the next rank begins with trip t_{i+1} on the same route-direction pair, the headway between them must still be counted. The implementation therefore uses `MPI_Sendrecv` so each rank can compare its first local trip to the previous rank's last trip.

Algorithm 1 Distributed Transit Analytics

Require: Sorted binary trip-record dataset; backend ∈ {cpu, cuda}
Open dataset collectively with MPI-I/O
Partition records across ranks
Read local block with `MPI_File_read_at_all`
Exchange one boundary record with neighboring rank
if backend = cuda **then**
    Copy local block to GPU
    Launch CUDA kernel to accumulate local statistics
else
    Accumulate local statistics on CPU
end if
Reduce local arrays and sums to rank 0
Emit timing, provider summaries, and optional CSV reports

4.5 CPU and CUDA Backends

The local analytics stage supports two interchangeable backends:

- **CPU backend**: MPI-distributed host-side accumulation for all statistics;
- **CUDA backend**: GPU-accelerated local analysis within each rank.

The CUDA path uses the rank's node-local communicator to select a GPU deterministically and then copies the local record partition to device memory. A CUDA kernel computes departures per time bucket, route-type counts, trip-duration sums, stop-count sums, and headway-related aggregates.

4.6 Global Reduction

Each rank computes local counts, sums, and squared sums. Rank 0 receives the global totals with `MPI_Reduce`, then derives:

- hourly departure profiles,
- peak/off-peak ratios,
- route-type counts,
- mean and coefficient of variation of headways,
- bunching and wide-gap rates, and
- trip-duration summaries.

Because the local arrays must be combined to produce provider-level outputs, the analysis necessarily includes a collective aggregation stage.

5 Experimental Methodology

All cluster experiments target AiMOS POWER9 nodes with NVIDIA V100 GPUs and use the module stack

```
module load xl_r spectrum-mpi cuda
```

as described in the course materials. The build process compiles the MPI logic with `mpicc` and the CUDA logic with `nvcc`, then links the resulting objects into a single executable.

The performance study follows the assignment requirements:

(1) **Strong scaling**: fix the total record count and increase the rank count.
(2) **Weak scaling**: fix the records per rank and grow the total dataset proportionally.
(3) **CPU-only MPI vs. hybrid MPI+CUDA**: compare the two backends directly.

Table 2: Experiment matrix used for the AiMOS timing study.

Study	Ranks	Requested Records
Strong scaling	2, 4, 8, 16	3,703,385 fixed
Weak scaling	2, 4, 8, 16	150,000 per rank

Each run reports:

- MPI-I/O time,
- boundary-exchange time,
- local compute time,
- reduction time, and
- total runtime.

We report this breakdown so we can separate bottlenecks in shared-file access, communication, and computation.

At this point, both the provider-analysis workflow and the AiMOS timing workflow are producing reportable outputs. The provider-analysis figures come from the local archive-backed binary dataset, while the timing figures come from the successful AiMOS experiment sweeps recorded in `experiment_results.csv`. The strong-scaling experiments use the full 3,703,385-record dataset. The weak-scaling experiments keep 150,000 records per rank and therefore grow from 300,000 records at two ranks to 2.4 million records at sixteen ranks.

6 Validation and Limitations

6.1 Validation Strategy

Because this project combines preprocessing, distributed execution, and GPU acceleration, correctness had to be checked at more than one stage. The first level of validation was at preprocessing time. For each provider group, the generated binary dataset was compared against simple feed-level expectations such as the number of routes retained, the number of trip records emitted, and whether basic schedule summaries were plausible. This step was especially important for feeds that did not follow the most common GTFS packaging pattern. Missing `calendar.txt` files, nested zip archives, and extended route-type values could all introduce silent errors if they were not handled explicitly.

The second level of validation was at the analytics stage. The provider-level results reported by the analyzer were checked for consistency across repeated runs and across backend choices. In particular, the CPU-only MPI backend and the hybrid MPI+CUDA backend were expected to produce the same aggregate outputs even when their runtime costs differed substantially. Matching provider summaries across those two paths served as a practical correctness check for the CUDA kernel, the device-memory transfers, and the final reduction logic. Distributed runs were also checked across different MPI rank counts to make sure that partitioning the dataset did not change the final totals.

Boundary conditions required separate attention because route headways depend on adjacent records that may fall on different ranks. Without an explicit boundary exchange, a distributed run could undercount or distort headways at partition edges. To guard against that, the implementation exchanges one record of boundary state with neighboring ranks and then recomputes the first local

headway using the previous rank's last record when appropriate. This is a small communication step, but it is necessary for correctness. Its effect is visible in the fact that headway-based provider summaries remain stable across rank counts rather than drifting as the partition shape changes.

6.2 Current Limitations

The main limitation of the current study is that it analyzes scheduled service rather than observed operations. GTFS static feeds describe planned trips, not whether vehicles actually ran on time, whether trips were canceled, or how passenger demand was distributed across the network. As a result, the headway and irregularity metrics in this report should be read as schedule-structure summaries, not as direct reliability measures. A provider may show wide scheduled gaps because it is designed around commuter or regional service, while a dense urban provider may show frequent short scheduled headways without necessarily delivering those headways in practice.

A second limitation is the way historical coverage is represented. To make the workload larger and more realistic, the preprocessing stage now incorporates archive-backed GTFS snapshots for several providers. That improves computational scale, but the current binary schema does not preserve an explicit snapshot timestamp or materialize trips by service date. In other words, the dataset is richer than a single-feed snapshot, but it still aggregates multiple schedule vintages into one trip-record collection. The provider findings should therefore be interpreted as schedule-structure summaries over the collected snapshots rather than as a single clean service period.

There are also modeling choices in the preprocessing stage that simplify the problem. The binary trip-record representation is intentionally compact and does not preserve every field that might be useful in later analyses. Likewise, route-type normalization is necessary to compare providers cleanly, but it compresses some of the variety present in extended GTFS route codes. Because the archive-backed dataset includes multiple schedule vintages, route counts in the provider-analysis section are best interpreted as snapshot-aware route keys rather than as a provider's present-day route map.

Finally, the GPU result remains limited by workload shape even after the dataset expansion. The local analytics kernel does benefit from GPU execution, but the full application remains dominated by fixed costs such as CUDA runtime initialization and host-device transfers. This means the present report captures an important hybrid-programming result, but not yet a case where the GPU wins end-to-end. Larger datasets, repeated analyses over resident device data, or heavier local kernels would likely change that balance. The current implementation therefore demonstrates a correct hybrid workflow more clearly than it demonstrates a production-ready acceleration point.

7 Transit Provider Findings

7.1 Peak vs. Off-Peak Concentration

Figure 2 compares the ratio of peak-period trip starts to off-peak trip starts for each provider. We define peak as 7–10 AM and 4–7 PM. NJ Transit again shows the largest concentration of service in those windows, with a peak/off-peak ratio of approximately 0.607. MTA follows at 0.551, while CTA, MBTA, and SEPTA cluster near 0.51–0.52. Even after adding archive-backed schedule snapshots, the pattern remains intuitive: NJ Transit is the most commuter-peaked system in the group, while the larger urban systems provide more all-day service.

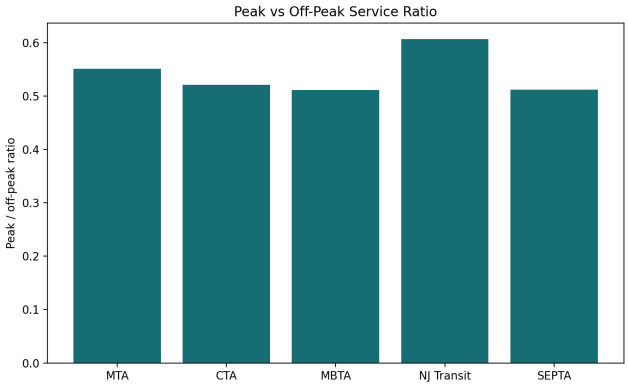

Figure 2: Peak/off-peak service ratio by provider, computed from local GTFS trip-start counts.

7.2 Hourly Service Profile

Figure 3 shows the hourly trip-start counts for each provider. Because the current dataset aggregates archive-backed snapshots, the absolute counts are much larger than in the earlier single-snapshot version of the report, but the shape of the curves is still informative. MTA and MBTA dominate in total scheduled volume, all providers show strong growth into the morning peak, and NJ Transit still drops off more sharply outside commuter windows. The archive-backed data therefore makes the workload larger without changing the broad service-shape story.

7.3 Service Span and Overnight Coverage

Hourly volume does not show how long each system stays active across the service day. Figure 4 compares the first scheduled departure and last scheduled arrival in each provider's GTFS records, measured in service-day hours so trips past midnight remain visible. MBTA and NJ Transit extend furthest into the late-night or early-morning tail, while CTA ends earliest at about hour 25.5. MTA starts at the beginning of the service day and runs to about hour 28.8, while SEPTA reaches nearly hour 27.9. This figure complements the hourly profile by showing overnight coverage directly.

7.4 Headway Distribution

Headways separate the providers more clearly than the peak/off-peak ratio alone. Figure 5 summarizes mean route headways by provider using quartiles over route-direction groups. CTA and MTA still have the smallest median headways at about 3.89 and 4.37 minutes. MBTA is close behind at 4.66 minutes, while SEPTA rises to 8.84 minutes and NJ Transit remains much larger at 20.53 minutes. The ordering fits the expected split between dense urban service and lower-frequency regional service.

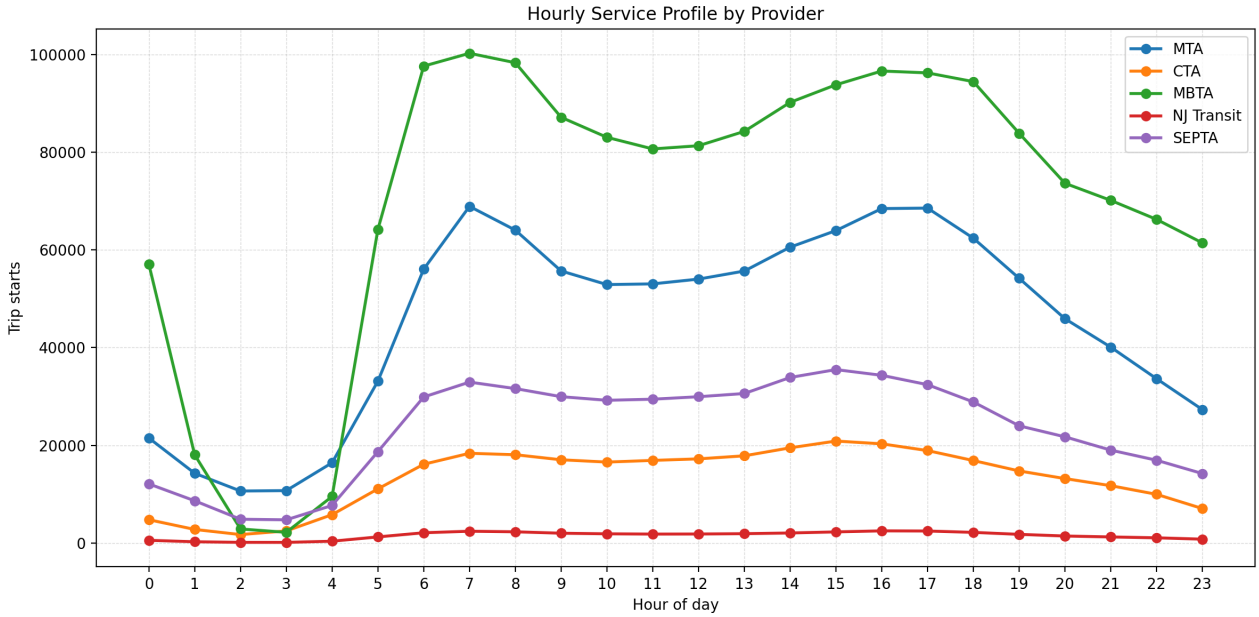

Figure 3: Hourly scheduled trip starts by provider.

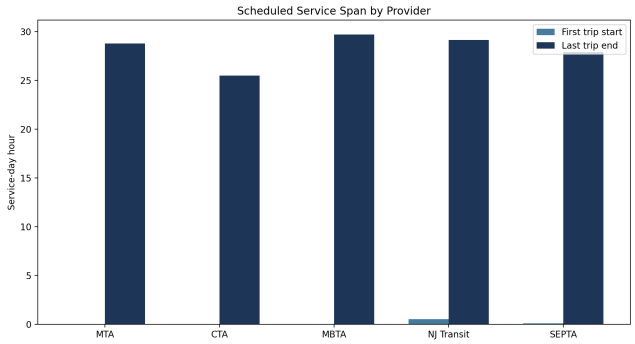

Figure 4: Scheduled service span by provider, measured in service-day hours so that overnight trips remain visible.

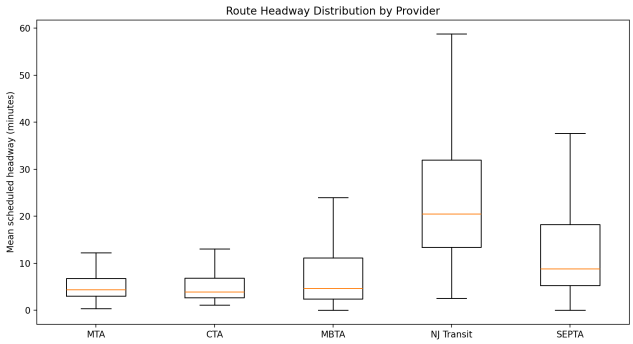

Figure 5: Distribution of route-level mean scheduled headways by provider.

7.5 Trip Duration Distribution

Figure 6 compares scheduled trip durations derived from GTFS start and end times. MBTA still has the lowest average duration at about 30.7 minutes, while MTA and CTA cluster near 50 minutes. NJ Transit remains the longest at 55.6 minutes on average, and SEPTA sits between the urban systems and the regional systems at roughly 41.0 minutes. The relative ranking is stable even after expanding the dataset with archived schedule snapshots.

7.6 Mode Composition

Figure 7 compares provider mode composition using normalized GTFS route types. Because the current dataset includes archived snapshots, the counts in this figure are snapshot-aware route keys rather than a present-day route map, but the relative mix is still useful. Bus routes dominate every provider group, MTA includes substantial subway and commuter-rail structure, MBTA has the broadest multimodal mix, and SEPTA remains more multimodal than CTA. Those differences help explain the headway patterns above.

7.7 Route Intensity

Absolute route counts can be misleading because a large system may either spread service across many routes or concentrate many trips on fewer corridors. Figure 8 therefore normalizes total trip starts by unique snapshot-aware route count. MTA and CTA remain the most intensive systems by this measure at roughly 640 and 610 trip starts per route, MBTA is lower at about 383, SEPTA is about 258, and NJ Transit is the least intensive at 131. This metric combines

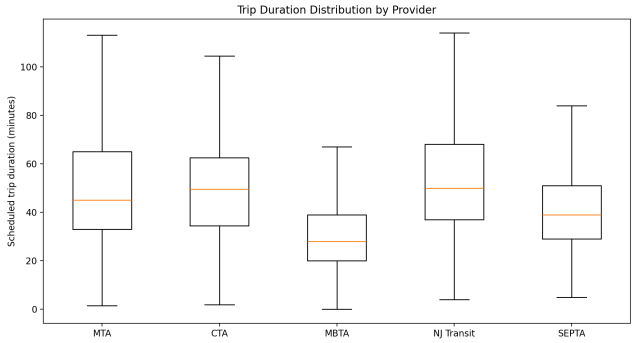

Figure 6: Distribution of scheduled trip durations by provider.

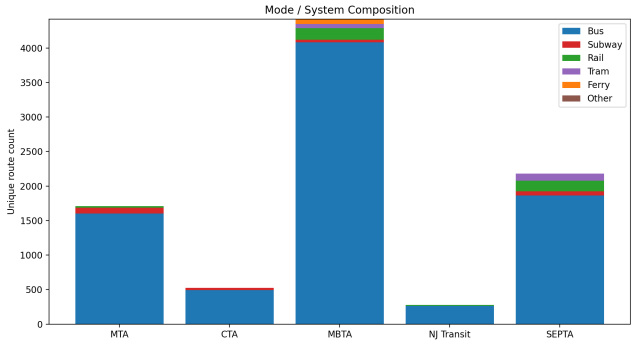

Figure 7: Provider mode composition after GTFS route-type normalization.

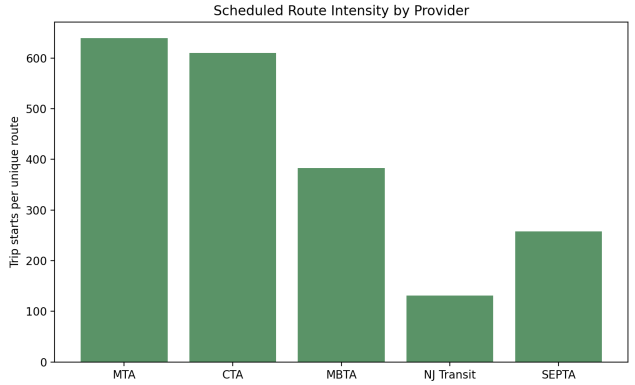

Figure 8: Scheduled route intensity by provider, defined as trip starts divided by unique route count.

route count with schedule density and helps explain why the urban systems maintain shorter median headways.

7.8 Headway Irregularity Proxies

The parallel analyzer also emits schedule-regularity proxies such as bunch-rate and wide-gap-rate summaries. These are not real-time reliability measures; they only describe how often scheduled

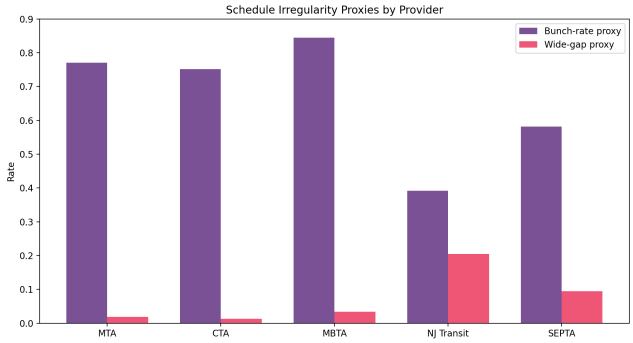

Figure 9: Schedule irregularity proxies emitted by the parallel analyzer.

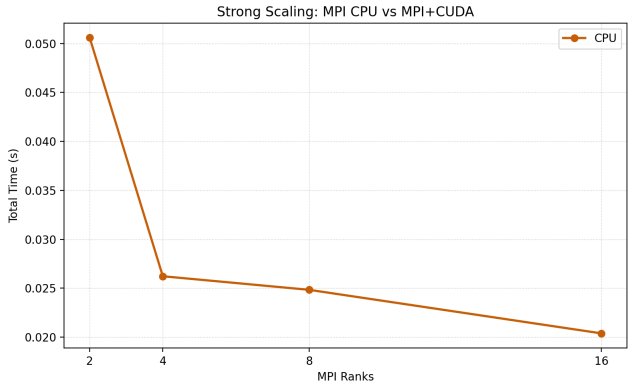

Figure 10: Strong scaling of the CPU-only MPI backend on the full archive-backed dataset.

headways fall below or above fixed threshold bands. Figure 9 shows a clear split between dense urban systems and regional systems. MBTA has the highest bunch-rate proxy at 0.845, followed by MTA and CTA, while NJ Transit remains the most wide-gap-heavy provider at 0.204. SEPTA sits between those extremes, with more gaps than the dense urban systems but much fewer than NJ Transit.

7.9 Provider Summary

Table 3 collects the main provider-side metrics used in this report. It also serves as a quick sanity check on the preprocessing output and makes it easier to compare the provider narratives above in one place.

8 Strong Scaling Parallel Performance

The figures in this section were generated from the AiMOS output stored in `experiment_results.csv`. The committed strong-scaling sweep completed for the CPU backend on the full archive-backed dataset, so the main question here is how much benefit MPI alone gets from additional ranks when the total workload is fixed at 3,703,385 records.

The strong-scaling study fixes the total record count at 3,703,385 records and increases the number of MPI ranks. The CPU-only

Table 3: Provider-level schedule summary derived from the local binary dataset.

Provider	Trip Starts	Peak/Off-Peak	Median Headway (min)	Avg Duration (min)	Trips/Route	Service Span (hr)	Unique Routes	Dominant
MTA	1,092,298	0.551	4.37	50.27	639.52	0.00–28.80	1,708	
CTA	319,822	0.521	3.89	49.62	610.35	0.00–25.50	524	
MBTA	1,693,567	0.512	4.66	30.70	383.25	0.00–29.72	4,419	
NJ Transit	36,483	0.607	20.53	55.60	130.76	0.50–29.17	279	
SEPTA	561,215	0.512	8.84	40.98	257.67	0.10–27.90	2,178	

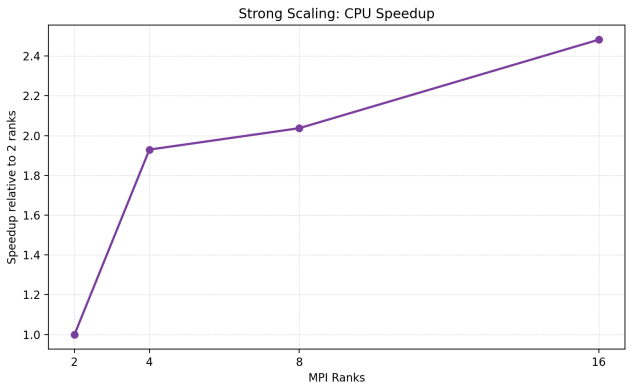

Figure 11: CPU strong-scaling speedup relative to the two-rank run.

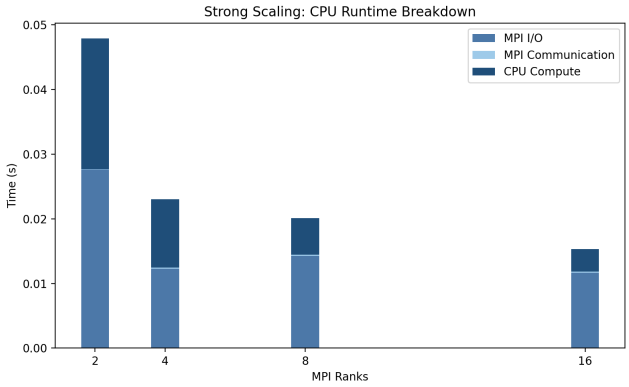

Figure 12: Strong-scaling runtime breakdown for the CPU-only MPI backend.

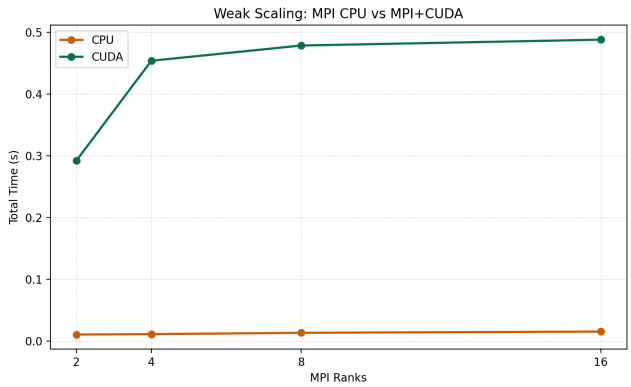

Figure 13: Weak scaling of the CPU-only MPI backend and the hybrid MPI+CUDA backend.

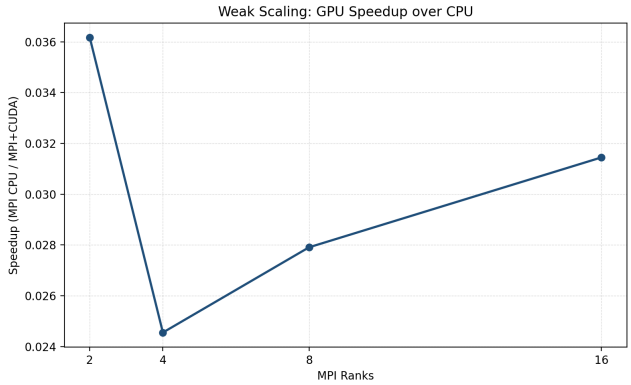

Figure 14: GPU speedup over the CPU-only MPI baseline under weak scaling.

MPI backend improves from 50.6 ms at two ranks to 26.2 ms at four ranks, 24.8 ms at eight ranks, and 20.4 ms at sixteen ranks. That is not ideal linear scaling, but it is still useful: the sixteen-rank run is about 2.48× faster than the two-rank run on the same full dataset.

Figures 10–12 show why the gains flatten. The local compute phase drops sharply from 20.2 ms at two ranks to 3.49 ms at sixteen ranks, but the I/O component stays in the 11–28 ms range and therefore becomes a larger fraction of the total runtime. Boundary exchange and reduction remain negligible. In other words, the CPU backend does benefit from more ranks, but the shared-file read and other fixed costs keep the scaling curve from being dramatic.

9 Weak Scaling Parallel Performance

The weak-scaling study fixes the records per rank and increases the total workload with the number of MPI ranks. In the successful runs, the CPU backend grows from 10.6 ms at 300,000 records and two ranks to 15.3 ms at 2.4 million records and sixteen ranks. That is a modest increase in runtime for an 8× increase in total work, so the weak-scaling behavior is reasonably good for the CPU path.

The CUDA backend behaves differently. Its weak-scaling runtime grows from 0.292 s at two ranks to about 0.488 s at sixteen ranks, and it remains far slower than the CPU baseline at every

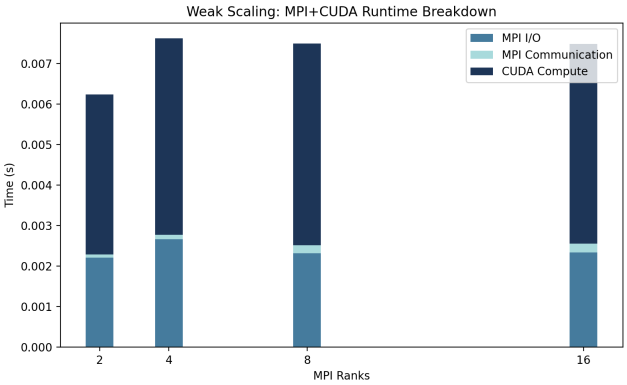

Figure 15: Weak-scaling runtime breakdown for the hybrid MPI+CUDA backend.

scale. As in the earlier experiments, the local GPU compute phase is only a few milliseconds, but the end-to-end application is still dominated by fixed CUDA overhead. The weak-scaling plots therefore reinforce the same architectural lesson as the summary run: the hybrid MPI+CUDA path is correct, but it is still overhead-bound for this workload.

10 Analysis of Performance Results

The measured results make the dominant bottlenecks clearer than the original design-level expectations:

- **CUDA runtime overhead dominates total time.** The measured GPU compute phase is only a few milliseconds, but total application time is hundreds of milliseconds, implying that device setup and transfer overhead are the primary bottleneck.
- **MPI-I/O matters more after the dataset expansion.** On the full strong-scaling runs, I/O stays in the 11–28 ms range and becomes a meaningful fraction of the total time even though it is still much smaller than the GPU end-to-end overhead.
- **Boundary exchange and reduction are negligible at current scale.** Both remain in the microsecond range, so they do not dominate either backend in the present dataset size.
- **The CPU baseline remains strong even on the larger dataset.** With 3,703,385 records total, the host-side loops are still inexpensive enough that GPU acceleration cannot amortize its fixed costs.

The main performance lesson is architectural. A hybrid MPI+CUDA design is not automatically faster than MPI on CPUs; it becomes faster only when the local compute intensity is high enough to amortize the GPU's fixed runtime costs. In the current implementation, that threshold has not been reached, so the GPU path is slower end-to-end even though the local kernel scales in the expected direction.

11 Discussion

The provider-level analytics show why this application is worth parallelizing. After adding archive-backed schedule snapshots, the dataset now contains more than 3.7 million trip records across five systems. New York contributes about 1.09 million records, Boston about 1.69 million, and Philadelphia about 561,000. A single local pass is still manageable, but the larger dataset makes the experimental results more defensible and gives the shared-file and reduction stages enough work to show up clearly in the timing breakdown.

The provider findings also line up with the service patterns one would expect. NJ Transit is still the most peak-concentrated and the most headway-sparse among the providers studied here, which fits a commuter-oriented system. CTA and MTA have the shortest route-level median headways, consistent with dense urban service, while SEPTA sits between those urban systems and NJ Transit. MBTA remains the shortest-duration system on average and also shows the highest bunch-rate proxy in the current aggregated schedule dataset. These are schedule-based findings, not passenger-based conclusions, but they still give a useful cross-city picture.

One practical lesson from the data-engineering stage is that publicly available GTFS feeds are cleaner in schema than in packaging. Missing `calendar.txt` files, nested archives, and extended route-type codes all had to be handled explicitly before the HPC work could begin. That preprocessing burden is easy to skip over in an algorithm sketch, but it matters in a system built on real feeds.

Another lesson is that no single metric is enough. Peak/off-peak ratios alone make the agencies look fairly similar, but route intensity, service span, headway distributions, and irregularity proxies separate them more clearly. MTA and CTA are both high-intensity systems by trips per route, yet MTA keeps that intensity across a longer service-day span. MBTA has a much shorter average trip duration than the commuter-oriented providers, while NJ Transit's peak concentration is easier to interpret when paired with its longer durations, larger headways, lower route intensity, and much higher wide-gap rate.

12 Team Contributions

Shimu Pan led the project from the data and report side. Early in the project, Shimu researched the GTFS format, identified which schedule files and derived metrics would be most useful for the transit-analysis portion of the assignment, and helped define the overall project scope and workflow. During implementation, Shimu built the preprocessing and dataset-construction pipeline that converts raw GTFS feeds into the fixed-width binary trip-record format used by the analyzer. This work included handling real feed irregularities such as missing `calendar.txt` files, nested archives, and extended route-type values, archive-backed feed acquisition, and nested SEPTA release packaging, as well as validating that the resulting provider summaries were plausible. In the later stages of the project, Shimu ran AiMOS experiments, collected and organized timing outputs, generated the figures used in the report, and integrated the final written report, including the narrative around the provider findings, performance results, and references.

Nikul Patel focused on the distributed MPI side of the system from design through testing. At the beginning of the project, Nikul researched the MPI and MPI-I/O features most relevant to the assignment, especially collective file access, partitioning strategies, and reduction patterns for distributed analytics. Nikul then implemented the MPI-based execution path, including record partitioning across ranks, collective input with `MPI_File_open` and `MPI_File_read_at_all`, the exchange of boundary state needed

for correct headway computation, and the global reduction logic that combines local statistics into provider-level results. Nikul also contributed to debugging and validation by checking that distributed runs produced consistent outputs across rank counts, helping verify timing breakdowns, and testing that the communication and reduction stages behaved correctly in the final analyzer.

Tim Liakh focused on the GPU and hybrid-execution side of the project. During the research and planning phase, Tim investigated how the CUDA programming model could be applied to the local analytics workload and how a GPU backend could be compared fairly against the CPU-only MPI baseline. Tim then implemented the CUDA kernels and supporting device-memory management needed to process local trip partitions on the GPU, including the movement of data between host and device and the accumulation of local analytics needed for the final reductions. As the project moved into evaluation, Tim helped test the CUDA path for correctness, compared CPU-versus-GPU behavior, and supported the interpretation of the final experimental results by identifying where GPU kernel acceleration was helping and where fixed runtime overhead was still dominating the end-to-end application.

13 Conclusions and Future Work

This project shows that GTFS schedule feeds can be turned into a parallel analytics workload using MPI, MPI-I/O, and CUDA. The binary trip-record representation supports collective shared-file access, the boundary-exchange step keeps headway calculations correct across partitions, and the CUDA path provides a direct comparison against CPU-only MPI. The provider-side results show clear differences in peak concentration, route headways, service span, trip duration, route intensity, and mode composition across MTA, CTA, MBTA, NJ Transit, and SEPTA, even after the dataset was expanded with archive-backed schedule snapshots.

The completed AiMOS timing study produced a clear result. On the 3,703,385-record full dataset, the CPU-only MPI backend improved from 50.6 ms at two ranks to 20.4 ms at sixteen ranks, and the weak-scaling runs stayed in a narrow 10.6–15.3 ms band while total work increased from 300,000 to 2.4 million records. The MPI+CUDA path runs correctly and its local kernel time scales in the expected direction, but the full application is still slower than CPU-only MPI because GPU initialization and transfer overhead dominate the end-to-end runtime. In this version of the workload, the GPU helps the kernel and hurts the total runtime.

Beyond the current prototype, the most promising extensions are:

- integrating `calendar_dates.txt` exceptions more explicitly,
- expanding `frequencies.txt` into generated trips when needed,
- adding GTFS-Realtime delay records to support true reliability analysis,
- experimenting with NVMe staging versus direct shared-file access, and
- extending the workload to more cities and longer time horizons.

With those additions, the project could grow from a schedule-analytics case study into a real-time transit-performance platform while still serving as a solid hybrid HPC workload.

References

[1] Saipraneeth Devunuri and Lewis Lehe. 2025. TransitGPT: A Generative AI-Based Framework for Interacting with GTFS Data Using Large Language Models. *Public Transport* 17 (2025), 319–345. doi:10.1007/s12469-025-00395-w

[2] S. Kiavash Fayyaz, Xiaoyue Cathy Liu, and Guohui Zhang. 2017. An Efficient General Transit Feed Specification (GTFS) Enabled Algorithm for Dynamic Transit Accessibility Analysis. *PLOS ONE* 12, 10 (2017), e0185333. doi:10.1371/journal.pone.0185333

[3] Philippe Fortin, Catherine Morency, and Martin Trépanier. 2016. Innovative GTFS Data Application for Transit Network Analysis Using a Graph-Oriented Method. *Journal of Public Transportation* 19, 4 (2016), 18–37. doi:10.5038/2375-0901.19.4.2

[4] Andrew Guthrie, Yingling Fan, and Kirti Vardhan Das. 2017. Accessibility Scenario Analysis of a Hypothetical Future Transit Network: Social Equity Implications of a General Transit Feed Specification-Based Sketch Planning Tool. *Transportation Research Record* 2671, 1 (2017), 1–11. doi:10.3141/2671-01

[5] Reyhane Javanmard, Luyu Liu, Jed A. Long, and Jinhyung Lee. 2025. Using Realtime GTFS to Generate Easy-to-Use Transit Accessibility Measures under Travel Time Uncertainty. *Travel Behaviour and Society* 41 (2025), 101054. doi:10.1016/j.tbs.2025.101054

[6] MobilityData. 2026. General Transit Feed Specification (GTFS) Overview. https://gtfs.org/documentation/overview/. Accessed 2026-04-19.

[7] MobilityData. 2026. General Transit Feed Specification Schedule Reference. https://gtfs.org/documentation/schedule/reference/. Revised March 3, 2026; accessed 2026-04-19.

[8] MobilityData. 2026. GTFS Realtime Reference. https://gtfs.org/spec/gtfs-realtime. Accessed 2026-04-19.

[9] MPI Forum. 2023. MPI 4.1 Standard Documents. https://www.mpi-forum.org/docs/mpi-4.1/. MPI 4.1 approved November 2, 2023; accessed 2026-04-19.

[10] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. 2008. Scalable Parallel Programming with CUDA. *Queue* 6, 2 (2008), 40–53. doi:10.1145/1365490.1365500

[11] NVIDIA. 2026. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html. Accessed 2026-04-19.

[12] Rafael H. M. Pereira, Marcus Saraiva, Daniel Herszenhut, Carlos Kaue Vieira Braga, and Matthew Wigginton Conway. 2021. r5r: Rapid Realistic Routing on Multimodal Transport Networks with R5 in R. *Findings* (2021), 21262. doi:10.32866/001c.21262

[13] Rajeev Thakur, William Gropp, and Ewing Lusk. 1999. Data Sieving and Collective I/O in ROMIO. In *Proceedings of the Seventh Symposium on the Frontiers of Massively Parallel Computation*. 182–189. doi:10.1109/FMPC.1999.750599

[14] Joseph Q. Zhou, Jay Jackson, Pam Stravitz, Gregory J. Barlow, and Peter Koonce. 2024. Addressing Data Latency in GTFS Realtime to Improve Transit Signal Priority. *Transportation Research Record* 2679, 1 (2024). doi:10.1177/03611981241255027